



O que vamos ver hoje ?

Javascript + APIs

Olá mundo

Parte I

- Programação Assíncrona em JavaScript
- Promises (ES6)
- Fetch API
- Async/Await (ES7/ES8)
- Browser APIs

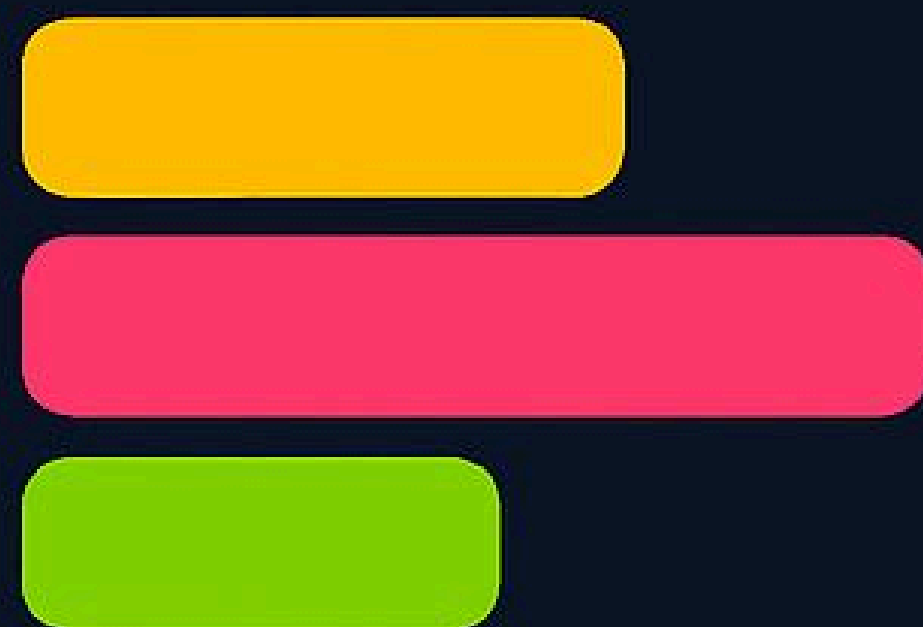


Programação Assíncrona em JavaScript

Os motores JavaScript são **single-thread**, o que significa que executam apenas uma tarefa por vez.

Se uma tarefa é demorada (como uma requisição de rede), ela bloquearia a execução de todo o código.

Uma **operação assíncrona** é uma operação que **não bloqueia** o fluxo de execução do programa. Ela permite que a thread principal continue executando enquanto a operação assíncrona é processada em segundo plano, geralmente fora da chamada principal



VS

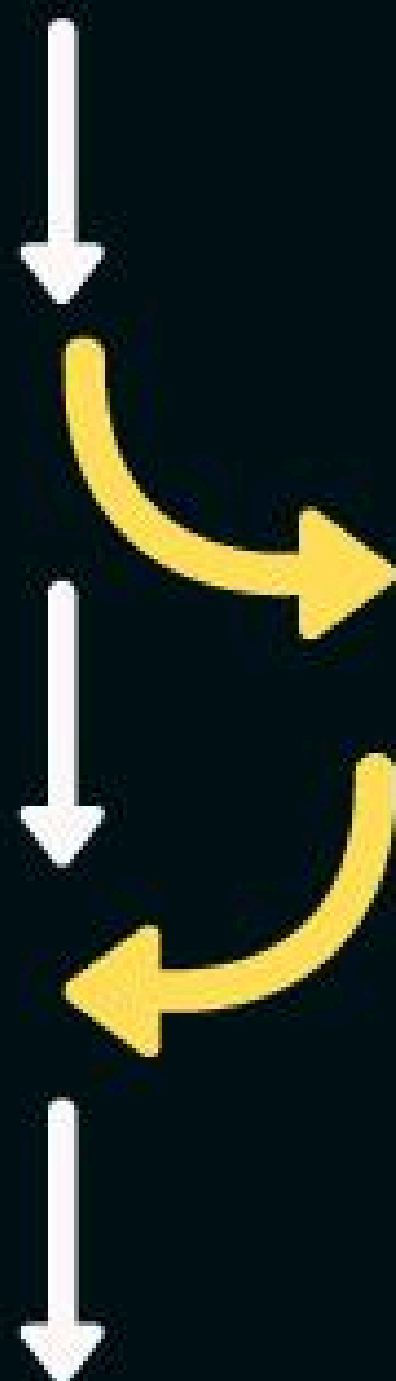


Asynchronous

Synchronous

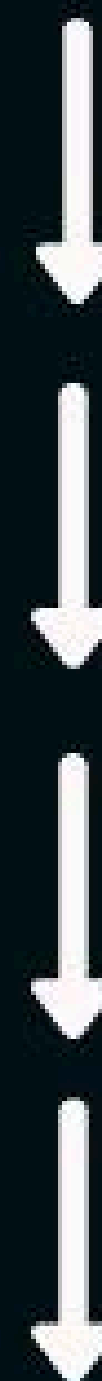


Asynchronous



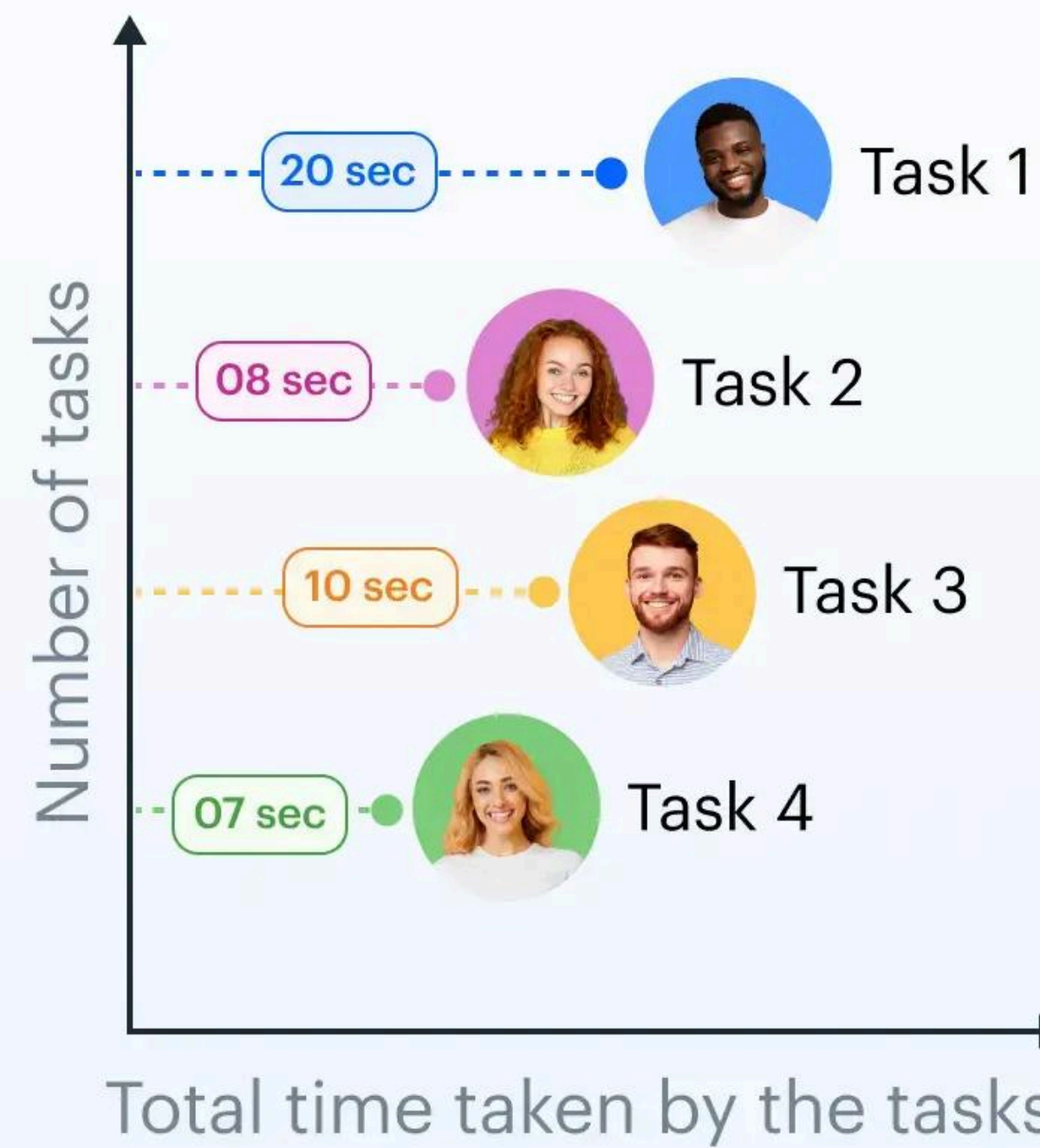
VS

Synchronous



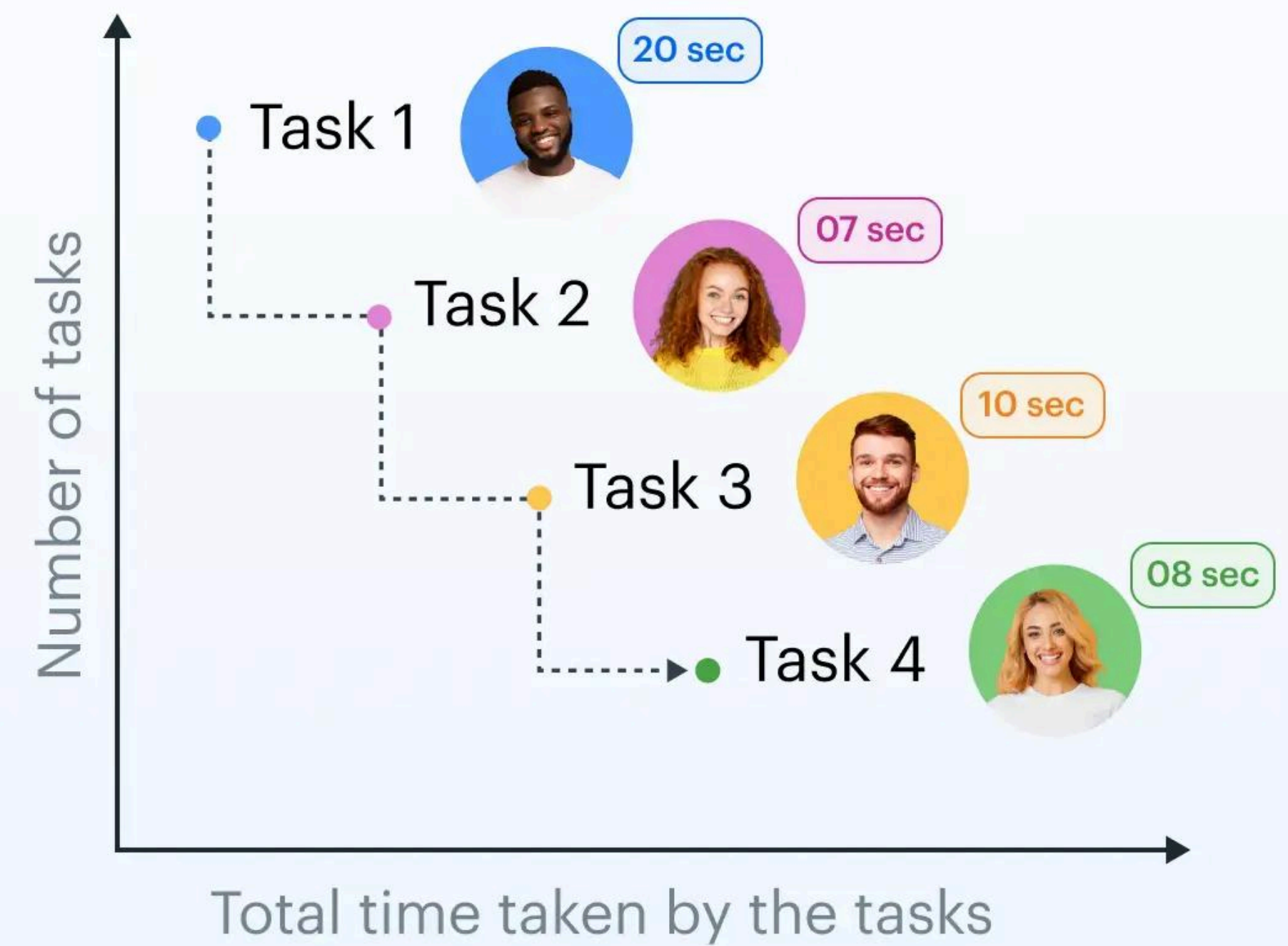


Asynchronous



VS

Synchronous





Programação Assíncrona

Ela permite que a thread principal continue executando enquanto a operação assíncrona é processada em segundo plano, geralmente fora da chamada principal

Callbacks

Promises

Fetch

Async/Await



Callbacks

Callbacks são funções passadas como argumentos para outras funções.

Funções assíncronas

frequentemente recebem uma função de callback como parâmetro para ser executada quando a operação for concluída.

```
1  // setTimeout() é uma função que executa uma função
   de callback após um tempo determinado.
2  setTimeout(function() {
3      console.log("Será exibida após 2 segundos.");
4  }, 2000);
5
6  // função chamando outra função de callback
7  function saudar(nome, callback) {
8      console.log('Olá, ' + nome + '!');
9      callback();
10 }
11
12 function despedir() {
13     console.log("Até logo!");
14 }
15
16 saudar("Maria", despedir);
17 // Olá Maria
18 // Até logo!
```




Promises (ES6)

Uma Promise é um objeto que representa a eventual conclusão (ou falha) de uma operação assíncrona.

Ela oferece uma forma mais organizada e limpa de lidar com código assíncrono do que os callbacks aninhados ("Callback Hell").

```
1  const minhaPromise = new Promise((resolve, reject) => {
2    // Simula uma operação assíncrona
3    setTimeout(() => {
4      const sucesso = true;
5      if (sucesso) {
6        resolve("Operação concluída com sucesso!");
7      } else {
8        reject("A operação falhou.");
9      }
10   }, 2000);
11 });
12
13 minhaPromise
14   // .then(): quando a Promise é resolvida.
15   .then(resultado => {
16     console.log(resultado);
17   })
18   // .catch(): quando a Promise é rejeitada (erro).
19   .catch(erro => {
20     console.error(erro);
21   });
```




Fetch API

A Fetch API é a interface moderna e baseada em Promises para realizar requisições de rede.

fetch(URL) retorna uma Promise. Para obter os dados em formato JSON, você deve chamar o método **.json()** na resposta, que retorna uma segunda Promise.

```
1  fetch('https://jsonplaceholder.typicode.com/posts/1')
2    .then((response) => {
3      return response.json();
4    })
5    .then((dados) => {
6      console.log(dados);
7    })
8    .catch((erro) => {
9      console.log('Ocorreu um erro:', erro);
10   });
```



Async/Await

Async/Await é uma forma mais prática para Promises.

async: Usado antes de uma função para indicar que ela contém código assíncrono.

await: "Pausa" a execução da função async até que a Promise seja resolvida.

```
1  const url = 'https://jsonplaceholder.typicode.com/  
  posts/1'  
2  
3  async function buscarDados() {  
4      const resposta = await fetch(url);  
5      const dados = await resposta.json();  
6      console.log(dados);  
7  }  
8  
9  buscarDados();
```



Browser APIs

As Browser APIs são interfaces que os navegadores expõem para o JavaScript interagir com o ambiente do navegador. Elas não são parte da linguagem em si, mas são acessíveis por meio dela. Algumas são padronizadas pela W3C.

Principais

document (DOM)

window

navigator (representa o navegador)

location (URL)

history (histórico de navegação)

Armazenamento

localStorage

sessionStorage

Outras APIs

Geolocation API

Clipboard API

Notification API



APIs de Armazenamento

localStorage: Armazena dados de forma persistente no navegador (os dados permanecem após o fechamento).

sessionStorage: Armazena dados apenas durante a sessão da página (até que a aba seja fechada).

```
1  // Salva um valor
2  localStorage.setItem('frutaPreferida', 'laranja');
3
4  // Recupera um valor
5  const fruta = localStorage.getItem('frutaPreferida');
6  console.log(fruta);
7
8  // Remove um valor
9  localStorage.removeItem('frutaPreferida');
10
11 // Salva um objeto serializado
12 const configuracoes = { tema: 'escuro' };
13 localStorage.setItem('config', JSON.stringify(configuracoes));
14
15 // Exemplo com Array
16 const listaCompras = ['leite', 'pão', 'café'];
17 // Converte o array para uma string JSON antes de salvar
18 localStorage.setItem('lista', JSON.stringify(listaCompras));
19
20 // Recupera a string JSON e a converte de volta para um array
21 const listaSalva = JSON.parse(localStorage.getItem('lista'));
22 console.log(listaSalva);
```




Outras APIs

Geolocation API: Permite acessar a localização do usuário.

Clipboard API: Permite copiar e colar conteúdo.

```
1  // Geolocation
2  navigator.geolocation.getCurrentPosition(
3    (posicao) => console.log(posicao),
4    (erro) => console.error(erro)
5  );
6
7  // Clipboard
8  navigator.clipboard.writeText('Texto a ser copiado.');
```

```
9  navigator.clipboard.readText().then(texto => {
10    console.log(texto);
11  });
```



Outras APIs

Notification API: Envia notificações para o usuário.

```
1  // Notification
2  // Verifica se o navegador suporta a API de Notificação
3  if ("Notification" in window) {
4      // Solicita permissão ao usuário para enviar notificações
5      Notification.requestPermission().then(permission => {
6          if (permission === "granted") {
7              // Se a permissão for concedida, cria uma nova notificação
8              const notificacao = new Notification("Título da Notificação", {
9                  body: "Esta é a mensagem da notificação.",
10                 icon: "https://placeholder.co/64x64/0056b3/ffffff?text=JS"
11             });
12
13             // Adiciona um evento de clique à notificação
14             notificacao.onclick = () => {
15                 console.log("A notificação foi clicada!");
16             };
17         } else {
18             console.log("Permissão para notificações negada.");
19         }
20     });
21 } else {
22     console.log("Este navegador não suporta a API de Notificação.");
23 }
```



Para saber mais:

https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Extensions/Async_JS

https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Extensions/Client-side_APIs/Introduction



Na próxima aula

Avaliação I

Obrigado